Published on *Journal of Intelligent Transportation Systems Technology, Planning, and Operations*
Volume 28, 2024 - Issue 6; Pages 956-970

Infrastructure Sensor-based Cooperative Perception for Early Stage Connected and Automated Vehicle Deployment

Chenxi Chen, Ph.D. Student
Department of Civil and Environmental Engineering
212 Sackett Building, University Park, PA 16802-1408
Email: chenchenxi@psu.edu

Qing Tang, Ph.D. Student
Department of Civil and Environmental Engineering
212 Sackett Building, University Park, PA, 16802-1408
Email: qingtang@psu.edu

Xianbiao Hu, Assistant Professor
Department of Civil and Environmental Engineering
212 Sackett Building, University Park, PA, 16802-1408
Email: xbhu@psu.edu

Zhitong Huang, PhD
Analysis, Modeling, Simulation program manager/Senior Transportation Research Scientist
Leidos Inc.
6300 Georgetown Pike, McLean, VA 22101
Email: zhitong.huang@leidos.com

ABSTRACT

Infrastructure-based sensors provide a potentially promising solution to support the wide adoption of connected and automated vehicles (CAVs) technologies at an early stage. For connected vehicles with lower level of automation that do not have perception sensors, infrastructure sensors will significantly boost its capability to understand the driving context. Even if a full suite of sensors is available on a vehicle with higher level of automation, infrastructure sensors can support overcome the issues of occlusion and limited sensor range. To this end, a cooperative perception modeling framework is proposed in this manuscript. In particular, the modeling focus is placed on a key technical challenge, time delay in the cooperative perception process, which is of vital importance to the synchronization, perception, and localization modules. A constant turn-rate velocity (CTRV) model is firstly developed to estimate the future motion states of a vehicle. A delay compensation and fusion module is presented next, to compensate for the time delay due to the computing time and communication latency. Last but not the least, as the behavior of moving objects (i.e., vehicles, cyclists, and pedestrians) is nonlinear in both position and speed aspects, an unscented Kalman filter (UKF) algorithm is developed to improve object tracking accuracy considering communication time delay between the ego vehicle and infrastructure-based LiDAR sensors. Simulation experiments are performed to test the feasibility and evaluate the performance of the proposed algorithm, which shows satisfactory results.

Keywords: Infrastructure sensors; cooperative perception; object tracking; time delay; unscented Kalman filter

1. INTRODUCTION

The development of connected and automated vehicle (CAV) technology relies on a set of onboard sensors to achieve safe and efficient operation. The sensor systems are composed of LiDAR, GNSS, cameras, radar, and ultrasonic sensors. The Automated Driving System (ADS) utilizes those onboard sensors and develops data fusion, data processing, and computing modules to accomplish detection, tracking, planning, and prediction tasks. The cost of sensors, especially LiDAR and GNSS, is relatively high at this stage and thus becomes a considerable barrier to the massive deployment of autonomous vehicle technology.

Infrastructure-based sensors, combined with cooperative driving automation (CDA) technology, provide a potentially promising solution to support the wide adoption of CAV technologies at an early stage. The benefits of infrastructure sensors are twofold. First, while higher-level vehicle automation is still under development and requires sophisticated sensors, many newer vehicles already have certain types of lower-level automation functions and connectivity. As infrastructure sensors perceive the state (e.g., location and speed) of mobility agents (e.g., vehicles, pedestrians, and bikers) in the vicinity area, the transmission of this additional knowledge to these lower-level CAVs will significantly boost its capability to understand the driving context and offer greater benefits in enhanced safety, as well as efficiency and energy reduction of the traffic flow. Further, even if a full suite of sensors is available on a vehicle with higher level of automation, occlusion and limited sensor range can pose significant challenges to safely navigating among pedestrians and other vehicles in the surrounding area, resulting in an increased risk of accidents and sub-optimal driving decisions (Orzechowski et al. 2018). Infrastructure-based sensors, on the other hand, when combined with cooperative perception technology, could significantly augment the capability in sensing and perception owing to the flexible location and angle for sensor installation (Jun and Markel 2017), especially in some challenging locations, for example, complicated urban or blind rural intersections.

Although promising significant benefits, the implementation of infrastructure sensors and their integration with the ego vehicle's perception do not come without a challenge. In this manuscript, a cooperative perception modeling framework is proposed to integrate the infrastructure sensors data and show how they may benefit a lower-level CAV without perception sensors, and a higher-level CAV with perception sensors,

respectively. In particular, the modeling focus is placed on a key technical challenge, time delay in the cooperative perception process, which is of vital importance to the synchronization, perception, and localization modules. In a cooperative perception system, the delay might be caused by multiple reasons. The first main component is the required time for the sensors to process the high-volume sensor data. For example Apollo, a popular self-driving software, is shown to take 200ms-550ms to detect a traffic light, and it has a 30% chance that the data processing will cost over 300ms (Luo 2019). Another significant component is the transmission delay when sharing the data between infrastructure sensors and vehicles (Allig and Wanielik 2019), which currently relies on Dedicated Short-Range Communications (DSRC) or Cellular-based Vehicle-to Everything (C-V2X) communication technologies (Petrov et al. 2021). For example, in a field testing regarding 4G LTE-based Vehicle-to-Cloud (V2C) communication, 65% of a total of 17,717 communication delay samples were in the range of 54ms-81ms (Liao et al. 2021). These numbers may vary with different software and different algorithms. In another experiment performed by Kim et al, the average data processing delay is 18.7ms and the communication delay for the first leader vehicle to the ego vehicle is 116.9ms with LiDAR- and compressed vision-based fusion (Kim et al. 2013). Such time delay may cause significant negative impacts on CAV operation. Due to communication latency, individual AVs will receive outdated information, which may result in inaccurate motion estimation of the moving objects and an increased risk of collision. Moreover, based on outdated information, the planning performed by vehicles may also deviate from the optimum choice and in the end degrade the performance of cooperative systems. Simple math can find that, for a vehicle driving at a speed of 70mph on a freeway, a latency of 100ms-250ms leads to 10ft-25ft error, which is rather significant.

To address these issues, this manuscript develops a real-time motion-estimation model to support cooperative perception by utilizing infrastructure-based sensors located at an intersection, or at any location along a generic roadway segment, where infrastructure sensors may detect additional information that onboard sensors may not. By incorporating explicit consideration of time delay and encompassing both longitudinal and lateral motion, this research aims to enhance the accuracy and effectiveness of motion estimation under cooperative driving scenarios for moving objects, including vehicles, cyclists, and

pedestrians, with explicit consideration of time delay, and in the end to enhance traffic safety and energy efficiency. To this end, a constant turn-rate velocity (CTRV) model is firstly developed to estimate the future motion states of a vehicle. A delay compensation and fusion module are presented next, to compensate for the time delay due to the computing time and communication latency. Last but not the least, as the behavior of moving objects (i.e., vehicles, cyclists, and pedestrians) is nonlinear in both position and speed aspects, an unscented Kalman filter (UKF) algorithm is developed to improve object tracking accuracy considering communication time delay between the ego vehicle and infrastructure-based LiDAR sensors. This proposed modeling framework can be applied to benefit both lower-level CAVs without perception sensors and higher-level CAVs with perception sensors.

To prove the feasibility and evaluate the performance of the developed model, we conduct a case study of cooperative perception at a signalized intersection, where a high-performance lidar is installed as an infrastructure sensor to collect data. The simulation results show that the information detected by infrastructure sensor is helpful for the operation of the vehicles both with and without onboard sensors, and the average motion estimation accuracy can be improved when delay compensation is considered. The proposed fusion model with explicit consideration of time delay can be generically applicable to the other types of infrastructure motion detection sensors, such as camera and radar.

This manuscript is organized as follows. Section 2 reviews relevant literature. Section 3 presents the proposed motion prediction methodology, including a motion model, delay compensation and fusion model, and UKF algorithm development. In Section 4, numerical analysis is implemented including case study and sensitivity analysis. Section 5 concludes this paper.

2. LITERATURE REVIEW

This section presents works related to detection and tracking using onboard or infrastructure-based sensors. We first review the detection and tracking methods in autonomous driving and then review the state of arts of perception methods in cooperative perception and infrastructure-based sensors. At last, the studies considering communication time delay are summarized.

With the development of the ADS, many studies focused on vehicle perception, based on the data from CAV sensors, among which object tracking is one of the most challenging parts. Numerous object tracking methods have been proposed. The Kalman filtering (KF) method (Blackman and Popoli 1999) is usually performed as standard technique to estimate a vehicle's state considering the uncertainties of the current state. KF is a special case of Bayesian filtering where the dynamic model and observation function are linear, and the uncertainty is represented using a normal distribution. Some studies focused on human detection (Schulz et al. 2001, Arras et al. 2008, Camara et al. 2020), while other works focused on detecting and tracking vehicles. For example, the tracking of the moving vehicles was implemented with KF and a bounding box was used to represent a vehicle in (Vu and Aycard 2009).

The popular KF-based vehicle kinematic models include the constant velocity model and constant acceleration model. These models assume that the velocity and acceleration are constant during the sampling time interval, and the noises are homogeneous in longitudinal and latitudinal directions, which is not realistic. Therefore, more realistic models such as constant turn rate and velocity model (CTRV) and constant turn rate and acceleration (CTRA) models were proposed (Blackman and Popoli 1999, Kaempchen et al. 2004, Polychronopoulos et al. 2007). These nonlinear models allow a vehicle to turn, and the motion prediction noise is added to the rotation and velocity independently. Consequently, the KF is modified to the Extended Kalman filter (EKF) (Wenzel et al. 2006, Jazwinski 2007), the Unscented Kalman Filter (UKF) (Wan and Van Der Merwe 2000, Julier and Uhlmann 2004, Antonov et al. 2011), and the particle filter (PF) (Bogdanski and Best 2018) for non-linear kinematic models. The main drawback for EKF is that since it approximates a non-linear system by the first-order linearization, it can result in large errors when high non-linearity occurs. The UKF can address the approximation issues of the EKF (Wan and Van Der Merwe 2000). It estimates the future state distribution with carefully chosen sample points by using the unscented transformation. Those sampling points, named sigma points, completely capture the posterior mean and covariance to the second order. Hence, the UKF can provide sufficient accuracy at equal computational costs and can be applied in many highly nonlinear filtering and control applications (Julier

and Uhlmann 2004). Antonov et al. (2011) applied the UKF to estimate vehicle state and found that the UKF outperformed the EKF, especially in the case of larger sampling times.

Since the infrastructure sensors can provide multiple benefits, such as augmenting the capabilities in sensing and perception and lowering the costs of CAV technology deployment, many studies focused on object detection and tracking based on infrastructure-based sensors. For example, Zhang et al. (2019a) developed an adjacent-frame fusion method for vehicle detection and tracking in complex traffic circumstances without object model extraction or a bounding box description. In (Wu et al. 2019), Wu et al. compared different methods for vehicle classification, such as Naïve Bayes, K-nearest neighbor classification, random forest (RF), and support vector machine, and they found that the decision tree-based method RF provided the highest accuracy. Next, Cui et al. (2019) applied an object tracking algorithm named global nearest neighbor (GNN) (Yin et al. 2007), recognizing the same vehicle in the current frame that has the minimum distance to the vehicle in the last frame. Furthermore, Zhao et al. (2019) presented a systematic approach to the detection and tracking of both vehicles and pedestrians at an intersection using infrastructure-based LiDAR sensors. They modified the DBSCAN method to improve the accuracy of object clustering. Then, they extracted three features and applied the backpropagation artificial neural network (BP-ANN) to distinguish pedestrians and vehicles and applied a discrete KF method for object tracking. An automatic background construction and object detection algorithm for 3D point clouds data was proposed (Zhang et al. 2019b) to expand the detection range and reduce the time complexity. To reduce the difficulty of creating a 3D point cloud dataset, a Grid-DBSCAN was developed (Zhang et al. 2020), containing gridding, clustering, and classification, to search the traffic objects and trained a CNN-based classifier to categorize the found objects by extracting the local features. With the goal of improving the clustering speed and addressing the problem of insufficient clustering and the under-segmentation, Gong et al. (2021) proposed a pedestrian detection algorithm by combining traditional and deep learning algorithms, in which octree with ROI selection is introduced to filter the background noise in each frame, and an improved Euclidean clustering algorithm was proposed for pedestrian detection.

Although infrastructure-based sensors provide augmented perception capability for CAVs, time delay exists in V2X communication due to the uncertain reliability of wireless communication links. Biswas et al. stated that communication delay may cause negative impacts on CAV cooperative control, including increased risk of collision and violation of system stability (Biswas et al. 2006). Some researchers studied the time delay to improve the cooperative tracking problem for the CAVs platoon. For example, considering identical and uniform communication delay, Gao et al. developed an H-infinity control method for a heterogeneous platoon to achieve the desired tracking performance and string stability requirement (Gao et al. 2016). Since vehicles are moving within a non-ideal wireless communication environment, information can be received by each vehicle with a different time-varying delay depending on the network conditions. Thus, identical communication delay for CAV platoons is a strict requirement. A few works considered time-varying delays, for example, Petrillo et al. (2018) considered multiple time-varying delays and proposed a decentralized adaptive strategy with the Lyapunov-Krasovskii approach to solving the leader tracking problem. The study addresses the time delay issue in a linear motion estimation system and proposed the use of a first-order filter to improve the synchronization of a platoon with a generic leader velocity profile. However, it is important to note that first-order filters assume a linear relationship between the current measurement and the previous estimated value, which may not hold true in all scenarios. Besides, Wang et al. (2021) proposed a consensus-based motion estimation approach to estimate the longitudinal motion of vehicles considering time-variant communication delay and package loss. However, in addition to the longitudinal motion, the lateral motion is also crucial in the context of autonomous driving and cooperative driving technology. In addition, communication delay also exists in the process of V2I communication. Several studies explored the probability density function (PDF) of V2I communication delay. Different PDFs were adopted to estimate the V2I communication delay, for example, normal distribution (Wang et al. 2021) and Gamma distribution (Ni et al. 2016). Fang et al. explored the characteristics of V2I communication delay and the impact on merging control at the on-ramp (Fang et al. 2022). Through field testing, they concluded that the distribution of V2I communication delay could be correlated with the application scenario and normal distribution can be generally adopted to approximate

the PDF when the sample size is large enough. To summarize, although time delay has been recognized in many above studies, its modeling and compensation for the infrastructure sensors-based cooperative perception have not been studied, which may lead to inaccurate motion estimation and increased collision risk. To address this research gap, the present study introduces a non-linear motion estimation model that explicitly considers time delay.

3. METHODOLOGY

3.1. Problem statement

In this section, we present a real-time motion-estimation model to support infrastructure sensors-based cooperative perception. Figure 1 illustrates a general study scenario, where an infrastructure-based sensor is installed on the side of a generic roadway segment. Three types of vehicles, namely higher-level CAV with perception sensors, lower-level CAV without perception sensors, and traditional vehicles that are neither connected nor automated, are present in the traffic flow. Pedestrians and bicyclists may also need to cross the road. An ego vehicle is a CAV that might be in lower-level or higher-level of vehicle automation, and receives data from infrastructure sensors that collect information on object vehicles which might be a CAV or a traditional vehicle. For example, if vehicle 1 is the ego vehicle, then vehicles 2, 3, 4, and 5 become the object vehicles, and the infrastructure sensor will detect the location and speed of these four vehicles, as well as those of pedestrians and bicyclists, and share them with vehicle 1 to guide its vehicle operation.

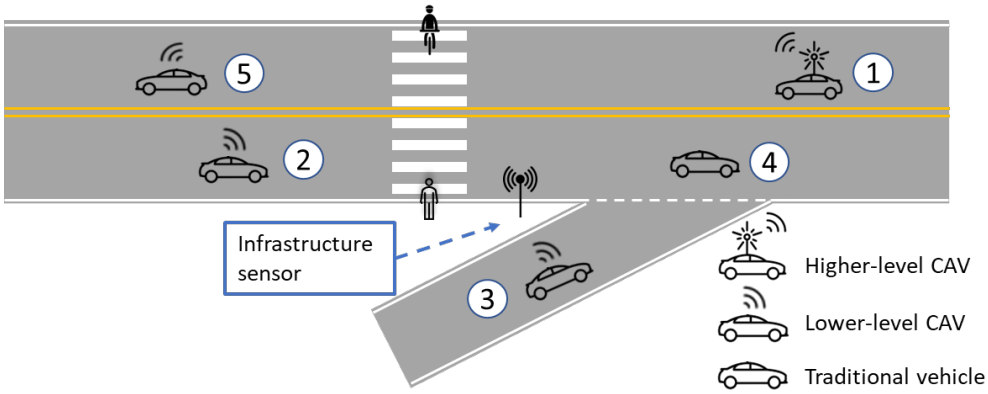


Figure 1 Cooperative perception scenario

3.2. Model framework overview

Figure 2 illustrates the framework of cooperative perception considering time delay. The input data of this research comes from the infrastructure sensors, and the ego vehicle with perception sensors (i.e., the higher-level CAVs). If the ego vehicle is a vehicle with lower-level of automation that does not have perception sensors, it can merely receive information but cannot sense the surrounding environment. Standard object detection from the Carla simulation model is used, which detects objects in the traffic flow and their motions and feeds them into the proposed real-time tracking model with three major components. The first component is a dynamic motion prediction model to predict the future motion state of a vehicle. The second component is designed to account for the time delay due to the computing time and communication latency. If the ego vehicle is a higher-level CAV, data fusion is also performed to collectively make use of the data from infrastructure sensors and CAV onboard sensors. If the ego vehicle is a lower-level CAV, then such fusion is not necessary, and we will rely solely on the data from the infrastructure sensors. The third component is the UKF algorithm, for the purpose of predicting the real-time trajectory, which will be described in detail in subsection 3.5.

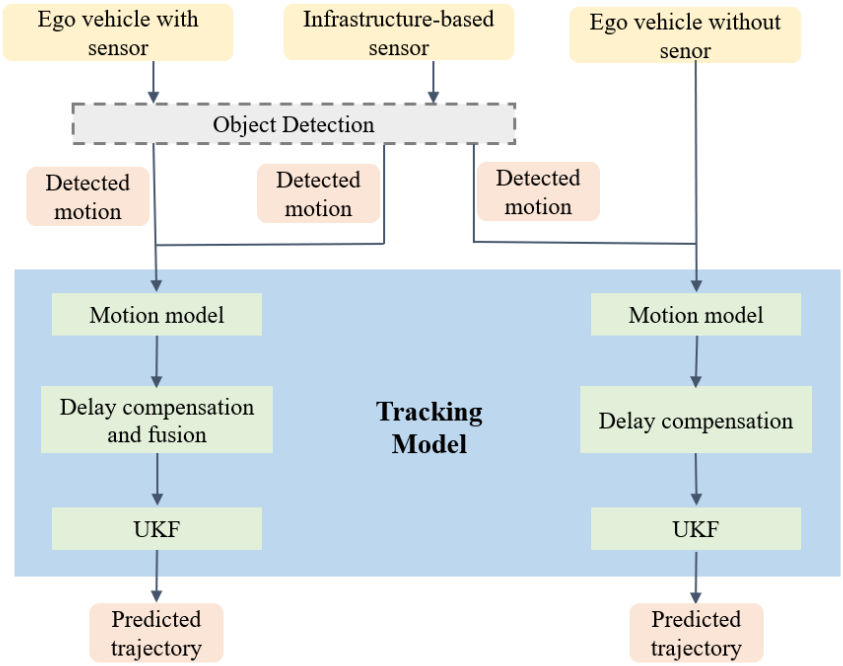


Figure 2 Cooperative Perception Framework Overview

As a convenient reference, the mathematical notations used in this section are presented below.

px_k	x direction position, subscript k means at time t_k, same below
py_k	y direction position
θ_k	yaw or heading
v_k	linear velocity
ω_k	angular velocity
$\boldsymbol{x}_k$	estimated state
χ_k^i	the ith sigma point
$\bar{x}_k$	mean of x_k
$\boldsymbol{P}_k$	covariance of x_k
Z_k^i	the ith updated sigma points
$\boldsymbol{z}_k$	measured state
$\boldsymbol{\sigma}_k$	motion noise

$\boldsymbol{\mu}_k$	measurement noise
Δt_k	time step between t_{k-1} and t_k
n	the length of x_k
λ	scaling parameter used in UKF
α	scaling parameter used in UKF
β	scaling parameter used in UKF
κ	scaling parameter used in UKF
W_m^i	weight for the mean of sigma point χ^i
W_c^i	weight for the covariance of sigma point χ^i
$\widehat{\boldsymbol{x}}_k^-$	mean of sigma points after prediction/priori
$\mathbf{P}_k^-$	covariance of sigma points after prediction
$\widehat{\boldsymbol{z}}_k^-$	mean of sigma points after updating/posteriori
$\mathbf{P}_k^+$	covariance of sigma points after updating
$\mathbf{Q}_k$	covariance of process noise regarding motion
$\boldsymbol{R}_k$	covariance of process noise regarding measurement
$\boldsymbol{\delta}_k$	residual of measurement $\boldsymbol{z}_k$
K_k	Kalman gain

3.3. Constant turn-date velocity-based motion model

The future motion states of a vehicle are estimated with a constant turn-rate velocity (CTRV)-based motion model described below. Instead of assuming vehicles move towards a straight line in the constant velocity model, CTRV increases a degree of freedom, yaw, or heading, and allows a vehicle to turn at a constant turn rate and speed during the sampling time interval. Therefore, the model is nonlinear, as shown in the following figure and equations.

The state space of the CTRV-based model is composed of position, orientation angle, and velocity. To be specific, the state space of a vehicle at time t_k is described by $\boldsymbol{x}(t_k)$ which can be computed via Eq. (1),

where px_k and py_k are the position in two dimensions and θ_k represents yaw. v_k and ω_k are the linear velocity and angular velocity respectively. The evolution of a tracking vehicle's state over time is presented in Figure 3. The state space can be transformed by the CTRV model which is composed of nonlinear state transition equations shown in Eq. (1), where $\Delta t_k = t_{k+1} - t_k$ is the sampling time interval. The vehicle is assumed to move with a constant turning rate ω_k and speed v_k in each short time interval Δt_k, thus, the turning radius is v_k/ω_k. At time t_{k+1}, the yaw turns to be $(\omega_k \Delta t_k + \theta_k)$ as shown in Figure 3. Therefore, state space of a vehicle at time t_{k+1}can be computed by Eq. (2).

$$\boldsymbol{x}(t_k) = (px_k, py_k, \theta_k, v_k, \omega_k)^T \tag{1}$$

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \Delta t_k) = \begin{pmatrix} \frac{v_k}{\omega_k}\sin(\theta_k + \omega_k \Delta t_k) - \frac{v_k}{\omega_k}\sin(\theta_k) + px_k \\ -\frac{v_k}{\omega_k}\cos(\theta_k + \omega_k \Delta t_k) + \frac{v_k}{\omega_k}\cos(\theta_k) + py_k \\ \omega_k \Delta t_k + \theta_k \\ v_k \\ \omega_k \end{pmatrix} \tag{2}$$

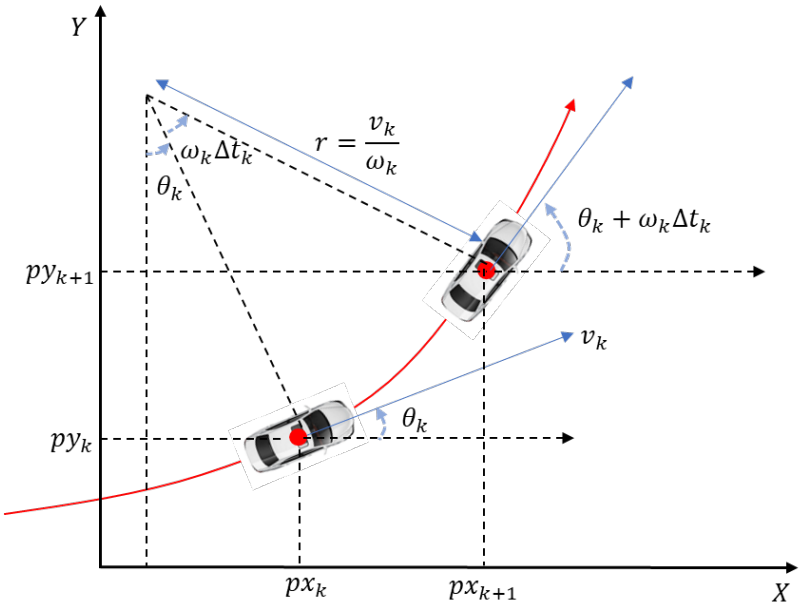


Figure 3. Evolution of tracking vehicle

3.4. Motion model-based delay compensation and fusion

This subsection describes the data compensation and fusion process, to account for the time delay due to the computing time and communication latency. As the motion state detected by the infrastructure-based sensor becomes outdated after transmission to the ego vehicles, we apply the motion model to compensate for the time delay. To distinguish the updated states $\boldsymbol{x}$, the measured states are represented by $\boldsymbol{z}$. Assuming the infrastructure-based sensor detects the object vehicle's dynamic states $\boldsymbol{z}_i^I$ at time t_i. The superscript I represents the motion is collected from an infrastructure-based sensor. Then the states will be disseminated to the ego vehicle, which receives the data with a time delay Δt_d. Consequently, the states at time $t_e = t_i + \Delta t_d$ can be predicted by the CTRV model in Eq. (3).

$$\boldsymbol{z}_e^I = f\left(\boldsymbol{z}_i^I, \Delta t_d\right) \tag{3}$$

Thus, the trajectory of the outdated motion collected by infrastructure sensors from time i_1 to i_n can be represented by $\left(\boldsymbol{z}_{i_1}^I, \boldsymbol{z}_{i_2}^I, \dots, \boldsymbol{z}_{i_n}^I\right)$ and will be updated to $(\boldsymbol{z}_{i_1+\Delta t_d}^I, \boldsymbol{z}_{i_2+\Delta t_d}^I, \dots, \boldsymbol{z}_{i_n+\Delta t_d}^I)$. Then, for those ego vehicles without sensors, the predicted motion will be directly used for tracking which we will discuss in the next subsection.

However, if the ego vehicle is equipped with sensors, data fusion is needed before going to the next step. Assuming the states at time e measured by ego vehicle is represented by $\boldsymbol{z}_e^E$ (the superscript E represents the motion is collected from the ego vehicle sensor), during time e_1 to e_n, the object trajectory collected by ego vehicle is $\left(\boldsymbol{z}_{e_1}^E, \boldsymbol{z}_{e_2}^E, \dots, \boldsymbol{z}_{e_n}^E\right)$. The fusion process will aggregate the two series of motions together in chronological order. Next, the fused trajectory $\left(\boldsymbol{z}_{e_1}^E, \boldsymbol{z}_{i_1+\Delta t_d}^I, \boldsymbol{z}_{e_2}^E, \boldsymbol{z}_{i_2+\Delta t_d}^I, \dots, \boldsymbol{z}_{e_n}^E, \boldsymbol{z}_{i_n+\Delta t_d}^I\right)$ will be filtered by UKF which will be discussed in the next subsection.

3.5. Unscented Kalman Filter-based tracking model

This subsection describes the last component of our developed real-time tracking model, which we focus most of our modeling efforts on. The UKF updates vehicle motion states step by step with the measured data and finally tracks the real-time trajectory of the object vehicle. For a motion state at a specific time, the UKF predicts the real-time motion state using the last step UKF-estimated motion state and the currently

measured motion state. The following subsections will introduce the detailed process of how UKF updates vehicle motion states from px_k to px_{k+1} with measured data $\boldsymbol{z}_k$.

Kalman Filtering is the analytical implementation of Bayes filtering, seeking to estimate the object states from its posterior density (Thrun et al. 2006). To be specific, the KF assumes that the object dynamic equation and posterior density at the previous time step follow Gaussian distribution. Moreover, it assumes the system and measurement function are linear so that the Gaussian distribution will be kept through the linear prediction and update process. However, the assumption of KF may not work well if the tracked object dynamics cannot be perfectly captured by the linear motion model. For example, the behavior of a vehicle is nonlinear in both position and velocity in our context. As such, UKF is adopted to realize motion tracking utilizing a nonlinear model. The UKF algorithm avoids computationally expensive linearization when compared with EKF and attempts to approximate the complicated distribution after propagation through the original non-linear function based on the so-called sigma points. The Gaussian distribution can still be recovered by assuming the weighted mean and covariance of sigma points as the mean and covariance of the approximated Gaussian distribution.

The motion tracking model can be formed as the UKF model with the state transition equation shown in Eq. (4) and the measurement function shown in Eq. (5). The prediction function is formed by the CTRV state transition equation $f(\boldsymbol{x}_k)$ described by Eq. (6), and is used to predict vehicle motion at t_k with estimated motion at t_{k-1}. T_k is the time interval between t_k and t_{k-1}. Motion noise is represented by $\boldsymbol{\sigma}_k$. The measurement update equation assumes the measured position px_k and py_k contain noise $\boldsymbol{\mu}_k$, which shows the relationship between estimated motion $\boldsymbol{x}_k$ and measurement $\boldsymbol{z}_k$. There are only two values that are measured as shown in (7): x_k and y_k. The heading θ computed by detection algorithm were found not reliable and robust. Therefore, it was not considered as a measured value.

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k) + \boldsymbol{\sigma}_k \tag{4}$$

$$\boldsymbol{z}_k = h(\boldsymbol{x}_k) + \boldsymbol{\mu}_k \tag{5}$$

$$f(\boldsymbol{x}_k) = \begin{pmatrix} \frac{v_k}{\omega_k}\sin(\omega_k T_k + \theta_k) - \frac{v_k}{\omega_k}\sin(\theta_k) + px_k \\ -\frac{v_k}{\omega_k}\cos(\omega_k T_k + \theta_k) + \frac{v_k}{\omega_k}\sin(\theta_k) + py_k \\ \omega_k T_k + \theta_k \\ v_k \\ \omega_k \end{pmatrix} \tag{6}$$

$$h(\boldsymbol{x}_k) = \begin{pmatrix} px_k \\ py_k \\ 0 \\ 0 \\ 0 \end{pmatrix} \tag{7}$$

The following subsections will introduce the detailed process of how UKF updates vehicle motion states, from $\boldsymbol{x}_{k-1}$ to $\boldsymbol{x}_k$, with measurement $\boldsymbol{z}_k$.

3.5.1. Sigma Point Sampling

This subsection describes the process of sigma point sampling. Although the motion noise and the initial state $\boldsymbol{x}_0$ both follow the normal distribution, as the state transition function is nonlinear, the predicted motion state $\boldsymbol{x}_1$ and the follow-up motions will not be subject to normal distribution anymore. Consequently, it becomes challenging to calculate the distribution of predicted motions. UKF uses a minimal set of carefully selected sample points to represent the state distribution, called sigma points, and the mean and covariance are assumed the mean and covariance of normal distribution. Assuming the normal-distributed vehicle dynamic state $\boldsymbol{x}_{k-1}$ at step $k-1$ has mean $\overline{\boldsymbol{x}}_{k-1}$ and covariance $\boldsymbol{P}_{k-1}$, then, a total of $2n+1$ sigma points χ will be generated by Eqs. (8)-(10).

$$\chi_{k-1}^{0} = \boldsymbol{x}_{k-1} \tag{8}$$

$$\chi_{k-1}^{i} = \boldsymbol{x}_{k-1} + \left(\sqrt{(n+\lambda)\boldsymbol{P}_{k-1}}\right)_i, \qquad i = 1, 2, \dots, n \tag{9}$$

$$\chi_{k-1}^{i+n} = \boldsymbol{x}_{k-1} - \left(\sqrt{(n+\lambda)\boldsymbol{P}_{k-1}}\right)_i, \qquad i = 1, 2, \dots, n \tag{10}$$

where $n = |\boldsymbol{x}_k|$ is the number of variables that needs to be estimated and it equals to 5 in this study, χ_{k-1}^{i} is the ith point at step $k-1$, $\left(\sqrt{(n+\lambda)\boldsymbol{P}_{k-1}}\right)_i$ is the ith row of the matrix square root of $(n+\lambda)\boldsymbol{P}_{k-1}$, and λ is a scaling parameter and will be discussed in the following equations.

3.5.2. Vehicle dynamic states prediction

This subsection describes the process of vehicle dynamic states prediction from step $k-1$ to k. The Sigma points are transformed through prediction function f in Eq. (6), as shown in Eq. (11). Then the mean $\hat{\boldsymbol{x}}_k^-$ and covariance $\mathbf{P}_k^-$ of the priori is calculated by Eq. (12) and Eq. (13), which means the approximations of the mean and covariance of the transformed sigma points.

$$\chi_k^i = f\left(\chi_{k-1}^i, \Delta t_k\right), \qquad i = 0, \ldots, 2n \tag{11}$$

$$\hat{\boldsymbol{x}}_k^- = \sum_{i=0}^{2L} W_m^i \, \chi_k^i \tag{12}$$

$$\mathbf{P}_k^- = \sum_{i=0}^{2L} W_c^i \left[\chi_k^i - \hat{\boldsymbol{x}}_k^-\right]\left[\chi_k^i - \hat{\boldsymbol{x}}_k^-\right]^{\mathrm{T}} + \mathbf{Q}_{k-1} \tag{13}$$

This formulation uses one set of weights W_m for the means, and another set W_c for the covariance. The weights for the mean of χ^i are computed by Eq. (14) and (16). The weights for the covariance are computed by Eqs. (15) and (16). Parameter λ is computed by Eq. (17).

$$W_m^0 = \frac{\lambda}{n+\lambda} \tag{14}$$

$$W_c^0 = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta) \tag{15}$$

$$W_m^i = W_c^i = \frac{1}{2(n+\lambda)}, \qquad i = 1, 2, \ldots, 2n \tag{16}$$

$$\lambda = \alpha^2(n+\kappa) - n \tag{17}$$

where α determines the spread of the sigma points around the mean value, κ is a secondary scaling parameter, and β is used to incorporate priori knowledge of the distribution of $\boldsymbol{x}$.

The choice of sigma points and weights cannot be proven that it is ideal for all nonlinear problems. But we can find the sigma points are proportional to the square root of the covariance matrix, which makes sigma points roughly proportional to σ. The term n leads to more sigma points and smaller weight values when the state space has a high dimension.

3.5.3. Measurement update

This subsection describes the process of updating the priori in measurement space $\boldsymbol{z}$. As the measurement function h is linear, there is no need to resample sigma points in this section. The updated sigma points are generated by Eq (18). Then the mean $\hat{\boldsymbol{z}}_k^-$ and covariance $\boldsymbol{P}_{\boldsymbol{k}}^+$ of these points are computed by the unscented transform Eqs. (19) and (20).

$$Z_k^i = h\left(\chi_k^i\right), \qquad i = 0, \ldots, 2n \tag{18}$$

$$\hat{\boldsymbol{z}}_k^- = \sum_{i=0}^{2n} W_m^i \, Z_k^i \tag{19}$$

$$\boldsymbol{P}_{\boldsymbol{k}}^+ = \sum_{i=0}^{2L} W_c^i \left[Z_k^i - \hat{\boldsymbol{z}}_k^-\right]\left[Z_k^i - \hat{\boldsymbol{z}}_k^-\right]^{\mathrm{T}} + \mathbf{R}_k \tag{20}$$

The Kalman gain is defined as Eq. (24) and the residual of measurement $\boldsymbol{z}$ is computed by Eq. (21). The Kalan gain of UKF can be simply seen as the ratio of $\mathbf{P}_{\boldsymbol{x_k z_k}}$ which is the belief in state and $\mathbf{P}_{\boldsymbol{z_k z_k}}$ which is the belief in measurement.

$$\boldsymbol{\delta}_k = \boldsymbol{z}_k - \hat{\boldsymbol{z}}_k^- \tag{21}$$

$$\mathbf{P}_{\boldsymbol{x_k z_k}} = \sum_{i=0}^{2L} W_c^i \left[\chi_k^i - \hat{\boldsymbol{x}}_k^-\right]\left[Z_k^i - \hat{\boldsymbol{z}}_k^-\right]^{\mathrm{T}} \tag{22}$$

$$\mathbf{P}_{\boldsymbol{z_k z_k}} = \sum_{i=0}^{2L} W_c^i \left[Z_k^i - \hat{\boldsymbol{z}}_k^-\right]\left[Z_k^i - \hat{\boldsymbol{z}}_k^-\right]^{\mathrm{T}} \tag{23}$$

$$\mathbf{K}_k = \mathbf{P}_{\boldsymbol{x_k z_k}} \mathbf{P}_{\boldsymbol{z_k z_k}}^{\boldsymbol{-1}} \tag{24}$$

Finally, we compute the new state estimate using the residual and Kalman gain, and the new covariance is computed as:

$$\boldsymbol{x}_k = \hat{\boldsymbol{x}}_{k-1} + \mathbf{K}_k \boldsymbol{\delta}_k \tag{25}$$

$$\mathbf{P}_k = \mathbf{P}_k^- - \mathbf{K}_k \boldsymbol{x}_k \mathbf{K}_k^T \tag{26}$$

With such a formulation approach, the state distribution at time t_k can be described by the normal distribution with mean $\boldsymbol{x}_k$ and covariance P_k.

4. NUMERICAL ANALYSIS

In this section, we perform simulation experiments to test the feasibility and evaluate the performance of the proposed algorithm. The experiment setup is shown in Subsection 4.1, and results and sensitivity analysis are discussed in Subsections 4.2 and 4.3.

4.1. Experiment setup

The proposed tracking method is implemented with Carla and Autoware co-simulation platform. Carla is an open-source simulator for autonomous driving research which provides a 3D driving environment and supports flexible specification of sensor suites, environmental conditions, full control of all static and dynamic actors, and map generation. Autoware is an autonomous driving simulator that includes all the functions that autonomous vehicles need, e.g., localization, perception, planning, and control, which can also be replaced or modified to validate researchers' models or algorithms. Autoware can easily cooperate with Carla through ROS bridge and offers a full stack autonomous driving simulation platform from hardware to software. In our experiment, the scenario was set up in Carla which also collected sensor data and shared them with Autoware. The simulator provides the trajectories of the ego vehicle and objects, which is seen as the ground truth data and can be used to evaluate the accuracy of the proposed tracking model and benchmark models. The perception module in Autoware was modified to validate our object tracking model. Specifically, the lidar detector process the point cloud data to detect objects and output their dynamic motions. The motions will then be used to test our algorithm.

A snapshot of the simulation scenario is shown in Figure 4. The studied road section has four lanes in total, as well as a signalized intersection. A high-performance lidar is installed to collect data, with a computer to process the collected data. An ego vehicle is driving toward the intersection and intends to make a left turn. The object that may conflict with ego vehicle could be a vehicle, cyclist, or/and a pedestrian. In this experiment, we select a moving vehicle as the detection object which drives from the right to the left. It is assumed to arrive at the intersection at the same time as the ego vehicle, and the signal is green for both vehicles. Thus, the ego vehicle needs to detect and yield to the object vehicle in the opposite lane. The real-

time motion states of the object vehicle are generated and recorded once it is detected by any one of the sensors. The total time delay including computing time and communication latency used in the experiment is 100ms. To evaluate the motion estimation models, the relative error is defined by the distance between the precepted trajectory and the ground-truth trajectory.

Two scenarios are tested based on the Carla environment. In the first scenario, the ego vehicle is a lower-level CAV and does not have sensors, and the process of object tracking is illustrated on the right side of Figure 2. In the second scenario, the ego vehicle is a higher-level CAV and is equipped with sensors, and the process of object tracking is illustrated on the left side of Figure 2. In each scenario, the trajectory of object vehicle calculated by different algorithms is generated. Each trajectory includes information about the position and speed in both the x and y directions at intervals of 0.5 seconds. Approximately 200 raw data points were recorded within a duration of 100 seconds.

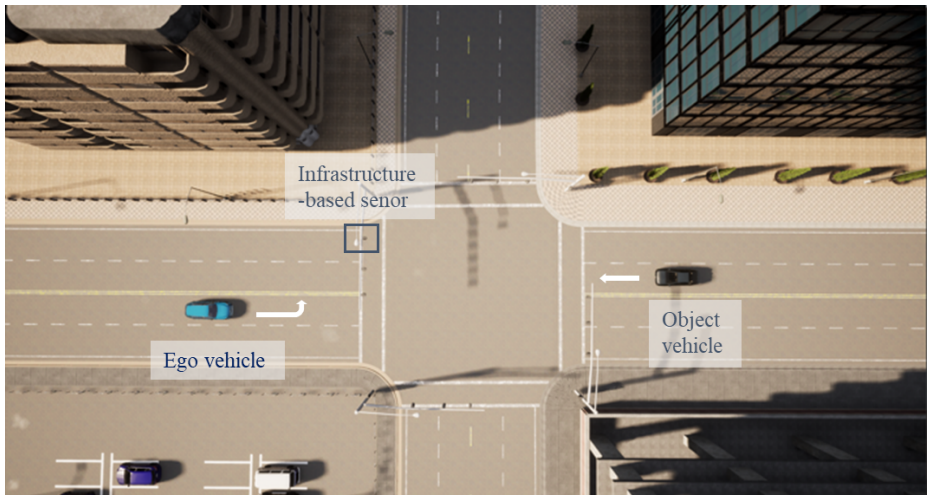


Figure 4 Experiment scenario

4.2. Results Analysis

4.2.1. Scenario 1: ego vehicle without perception sensor

In scenario 1, the ego vehicle is a lower-level CAV and does not have perception sensors. It receives the outdated detected motion state of the object transmitted from infrastructure-based sensors. The motion state of the object is estimated by the CTRV motion model and the measured motion state, to compensate the

time delay. Then, the measured motion state of the object after delay compensation serves as an input of the UKF. Another input is the motion state estimated by the CTRV motion model using the real-time motion state at the previous timestep, which is the output of the UKF at the last step. As a result, the real-time motion state at the current timestep is predicted using the UKF. We will compare the object tracking results using infrastructure-based data with delay compensation or without. Figure 5 shows the estimated trajectory when compared with the ground truth trajectory which represents the real trajectory of the object vehicle. The entire timeline spans 80s, with a total range in the x direction of approximately 250m. The estimated trajectory closely aligns with the ground truth trajectory. As such, we focus on the trajectory difference specifically during the 35s to 39s interval for a clearer presentation of the disparities. We can find that there is little difference between the ground-truth trajectory and estimated trajectory when the time delay is compensated, whereas there is more than 1m error if the time delay is ignored. The detailed estimation error is shown in Figure 6. We can find the error fluctuates between 1.05m and 1.62m if the time delay is not compensated, which is very significant and may cause safety concerns in reality. The average motion estimation error is 1.28m. Once we compensated the delay, the motion estimation error drops significantly and ranges from 0.13 m to 0.33m. The average motion estimation error is 0.22m. The average motion estimation error is reduced by 82.8% after delay compensation.

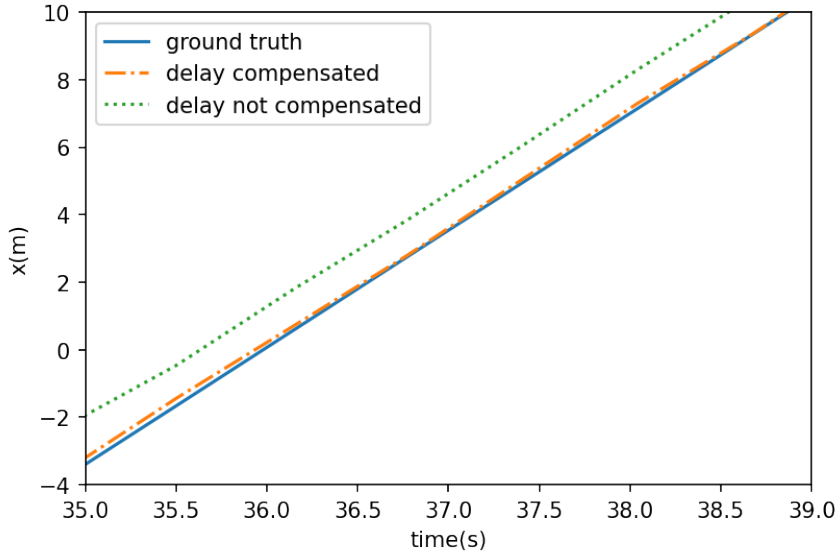


Figure 5 Ground truth and estimated trajectory on x direction in scenario 1

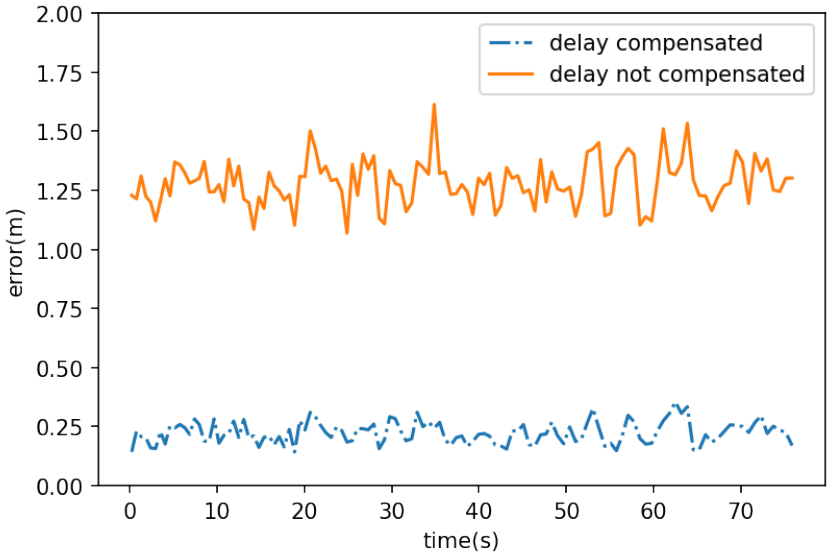


Figure 6 Motion estimation error comparison between delay is considered and not considered

4.2.2. Scenario 2: ego vehicle with sensor equipped

In scenario 2, the ego vehicle is a higher-level CAV and has perception sensors, as such, the object vehicle is detected by both onboard sensors and infrastructure-based sensors. The motion states detected by infrastructure sensors are transmitted to the ego vehicle with time delays. The CTRV motion model is applied to compensate for time delays. Then the detected motion states from onboard sensors and those from infrastructure sensors after delay compensation are fused as inputs of the UKF. We then compare the results with ground truth data, and the results are shown in Figure 7. There are three estimated trajectories provided by onboard sensors only (orange line), vehicle-infrastructure cooperative perception with delay compensation (green line), and without compensation (red line). We can find that the predicted trajectory by cooperative perception with delay compensation is the closest to the ground truth trajectory.

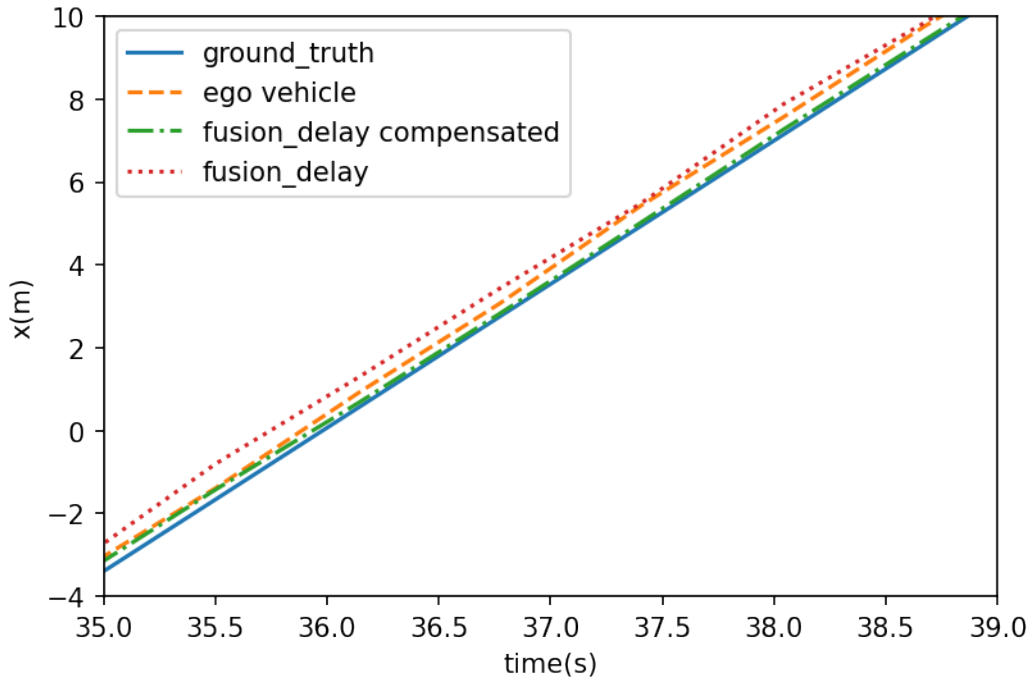


Figure 7 Ground truth and estimated trajectory on *x* direction in scenario 2

To analyze the performance of cooperative perception, Figure 8 illustrates the motion estimation error comparison between ego vehicle perception and cooperative perception. We can find the errors with ego vehicle perception fluctuates between 0.18 m and 0.65 m, with an average motion estimation error of 0.45m. In the cooperation scenario, the motion estimation error drops significantly to 0.11m-0.32m, with an average error of 0.2m. We can then conclude that cooperative perception improves the average motion estimation accuracy by 55.6%. Moreover, as shown in Figure 8, the ego vehicle can only detect objects between the 20s and 65s, whereas if onboard sensor data and infrastructure sensor data are fused, the perception range is increased to 0-80s. In other words, fusion with the infrastructure sensor data allows the ego vehicle to expand its perception range by 35s, or 77.8%, which is rather significant.

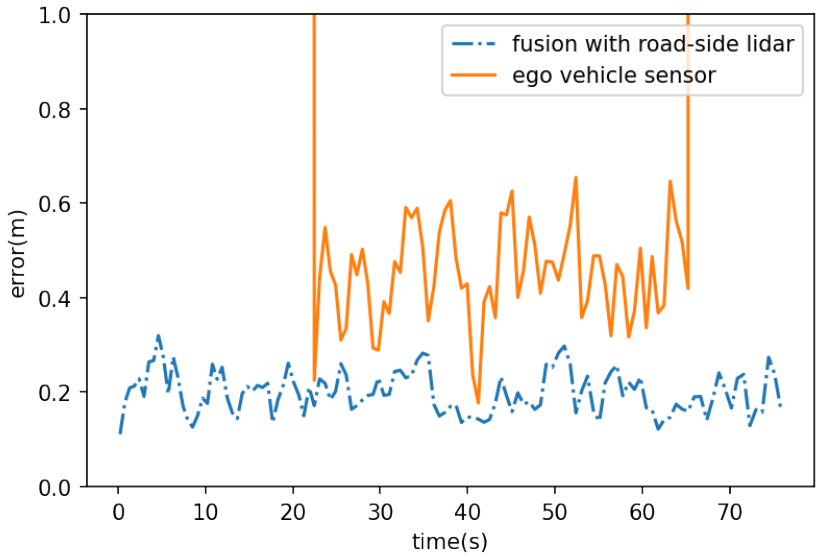


Figure 8 Motion estimation error comparison between ego vehicle-based perception and cooperative perception

Figure 9 illustrates the motion estimation error comparison between fusion with delay compensation and without compensation. We notice that the estimation errors without delay compensation fluctuate between 0.47m and 0.98m, with an average error of 0.7m, whereas the errors range from 0.11m to 0.32m if the time delay is compensated and the average error drops to 0.2m. Thus, the average motion estimation accuracy is improved by 71.4% with delay compensation.

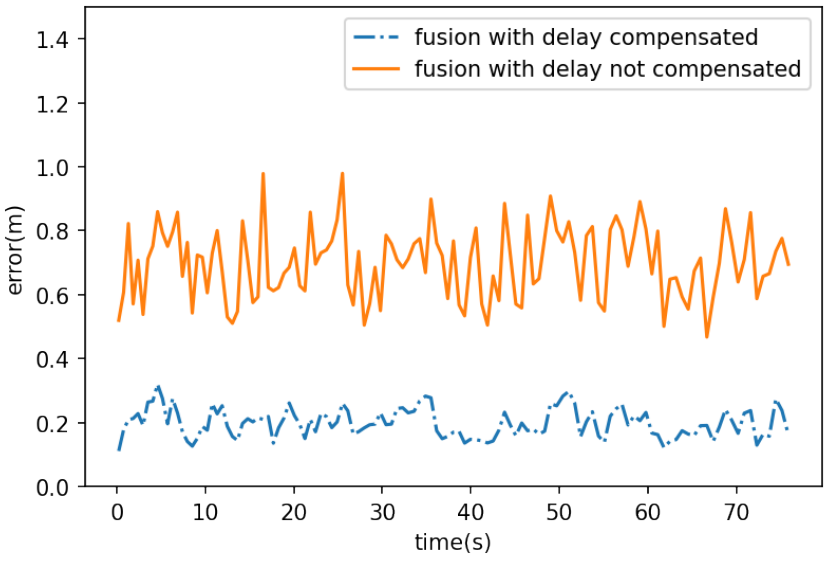


Figure 9 Motion estimation error comparison between fusion with delay considered and not considered

4.3. Sensitivity Analysis

In this section, we perform a sensitivity analysis to show how changes in the latency, the ego vehicle's speed, and the object's speed can influence the motion estimation accuracy. This sensitivity analysis starts with a set of baseline variables, in which the time delay is set to be 100ms, and the speed of ego vehicle and object vehicle are 20 mph. Each set of parameters was repeated 10 times in the same scenario as shown in Figure 4 and the average error was calculated.

4.3.1. When latency changes

First, we test what may happen if the latency changes from 50ms to 200ms. Figure 10 illustrates the motion estimation errors of ego vehicle perception only, cooperative perception with delay compensation, and cooperative perception without delay compensation, represented by blue, orange, and green curves, respectively. It can be observed that the average motion estimation error is 0.42m if using ego vehicle perception only, while the average error drops to 0.24m by 42.8% if cooperative perception with delay compensation. Moreover, the results indicate the average error increases significantly from 0.48m to 1.85m if the delay is ignored, as the delay increases from 50ms to 200ms. The curve for fusion with delay compensation performs the best. This result aligns with our understanding, indicating that as the delay increases, the motion estimation error for the object also increases when delay compensation is not taken into account. However, after considering delay compensation, the error is hardly affected. In other words, the higher the delay, the more crucial delay compensation becomes for the fusion of infrastructure-based sensor and onboard sensor.

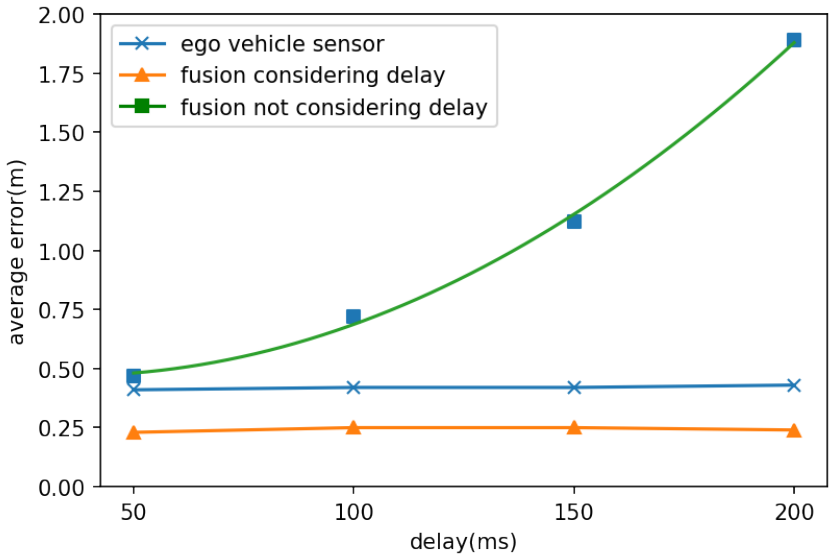


Figure 10 Motion estimation errors when the latency changes

4.3.2. When the ego vehicle's speed changes

Then, we test what would happen if the ego object's speed increases. Figure 11 illustrates the motion estimation errors of ego vehicle perception only scenario and cooperative perception scenario with delay compensation when the object vehicle speed changes from 10m/s to 50m/s. Both errors are found to increase with increased ego vehicle speed, however by different degrees. The results show that the motion estimation accuracy of the ego vehicle perception only scenario is more sensitive to the object speed, and the average error increases sharply from 0.33m to 0.62m by 87.9%. If performing cooperative perception considering delay, the average error is obviously lower than that of ego vehicle perception only scenario and increases slightly from 0.26m to 0.28m by 7.7%. The results indicate that cooperative perception can improve the motion estimation accuracy and will be barely affected by the speed of ego vehicle. This leads us to conclude that cooperative perception is of utmost importance in achieving higher accuracy in motion estimation, particularly on freeways and highways.

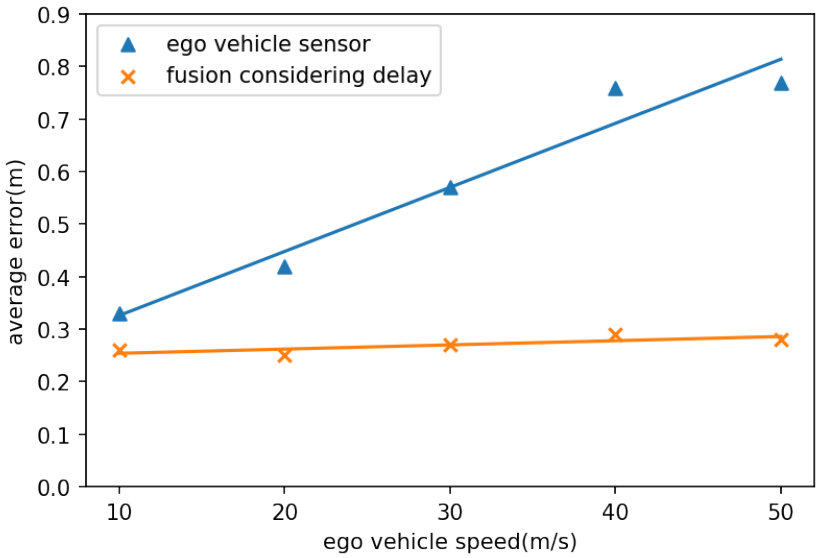


Figure 11 Motion estimation errors when the ego vehicle speed changes

4.3.3. When the object's speed changes

Finally, we test what would happen if the object's speed increases. Figure 12 illustrates the motion estimation errors of ego vehicle perception only scenario and cooperative perception scenario with delay compensation when the object vehicle speed changes from 10m/s to 50m/s. Both curves are shown to increase with increased speed. We can observe that the motion estimation accuracy of the ego vehicle perception only scenario is more sensitive to the object speed, and the errors increase significantly from 0.33 m to 0.62 m. If performing cooperative perception, the errors increase slightly from 0.21 m to 0.28 m as the object speed increases from 10 m/s to 50 m/s. It can be found that the motion estimation error drops by 36.4% when the object speed is 10m/s, while it dropped significantly by 54.8% as the speed increases to 50m/s. In light of these results, we can also conclude that cooperative perception plays a vital role in attaining higher accuracy in motion estimation, especially in the context of freeways and highways.

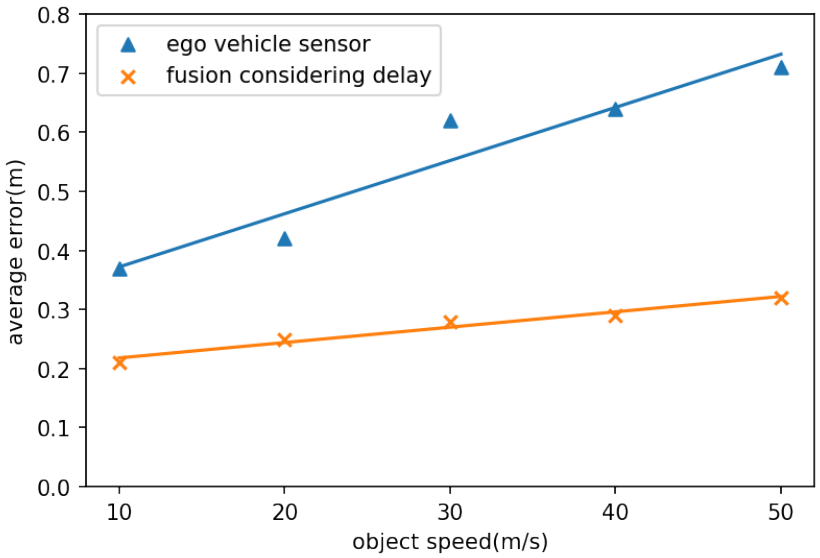


Figure 12 Motion estimation errors when the object speed changes

5. CONCLUSIONS

In this manuscript, a cooperative perception modeling framework is developed to integrate the infrastructure sensors data and show how they may benefit a vehicle with different levels of automation. In particular, the modeling focus is placed on a key technical challenge, time delay in the cooperative perception process, which is of vital importance to the synchronization, perception, and localization modules. The proposed models are implemented with Carla and Autoware co-simulation platforms. The results show when the ego vehicle is a lower-level CAV without onboard sensors, the information captured by infrastructure sensors can be successfully applied to guide the vehicle operation, and when delay compensation is considered, the average motion estimation accuracy is improved by 82.8%. For the high-level CAV case with perception sensors, after the fusion of the onboard sensor data and infrastructure data, the average motion estimation accuracy is improved by 71.4%, and the data fusion allows the ego vehicle to expand its perception range by 78%.

A future direction of this research involves the implementation of the proposed cooperative perception method on real CAVs and conducting field tests in real-world environments. This will include designing additional testing scenarios involving lane changing and turns to further assess the performance of the

method. To enhance the fidelity of the algorithm, it is essential to incorporate real vehicle dynamics rather than relying on the simplified version used in the Carla simulation platform utilized in this study.

6. AUTHOR CONTRIBUTIONS

The authors confirm their contribution to the paper as follows: study conception and design: Chenxi Chen, Xianbiao Hu, Zhitong Huang; data collection: Chenxi Chen; analysis and interpretation of results: Chenxi Chen; draft manuscript preparation: Chenxi Chen, Qing Tang, Xianbiao Hu, Zhitong Huang. All authors reviewed the results and approved the final version of the manuscript.

7. REFERENCES

Allig, C. and G. Wanielik (2019). Alignment of perception information for cooperative perception. *2019 IEEE Intelligent Vehicles Symposium (IV)*, IEEE.

Antonov, S., A. Fehn and A. Kugi (2011). Unscented Kalman filter for vehicle state estimation. *Vehicle System Dynamics*. 49(9): 1497-1520.

Arras, K. O., S. Grzonka, M. Luber and W. Burgard (2008). Efficient people tracking in laser range data using a multi-hypothesis leg-tracker with adaptive occlusion probabilities. *2008 IEEE International Conference on Robotics and Automation*, IEEE.

Biswas, S., R. Tatchikou and F. Dion (2006). Vehicle-to-vehicle wireless communication protocols for enhancing highway traffic safety. *IEEE Communications Magazine*. 44(1): 74-82.

Blackman, S. S. and R. Popoli (1999). *Design and analysis of modern tracking systems*, Artech House Publishers.

Bogdanski, K. and M. C. Best (2018). Kalman and particle filtering methods for full vehicle and tyre identification. *Vehicle System Dynamics*. 56(5): 769-790.

Camara, F., N. Bellotto, S. Cosar, F. Weber, D. Nathanael, M. Althoff, J. Wu, J. Ruenz, A. Dietrich and G. Markkula (2020). Pedestrian models for autonomous driving part ii: high-level models of human behavior. *IEEE Transactions on Intelligent Transportation Systems*. 22(9): 5453-5472.

Cui, Y., H. Xu, J. Wu, Y. Sun and J. Zhao (2019). Automatic vehicle tracking with roadside LiDAR data for the connected-vehicles system. *IEEE Intelligent Systems*. 34(3): 44-51.

Fang, Y., H. Min, X. Wu, W. Wang, X. Zhao and G. Mao (2022). On-Ramp Merging Strategies of Connected and Automated Vehicles Considering Communication Delay. *IEEE Transactions on Intelligent Transportation Systems*.

Gao, F., S. E. Li, Y. Zheng and D. Kum (2016). Robust control of heterogeneous vehicular platoon with uncertain dynamics and communication delay. *IET Intelligent Transport Systems*. 10(7): 503-513.

Gong, Z., Z. Wang, B. Zhou, W. Liu and P. Liu (2021). Pedestrian Detection Method Based on Roadside Light Detection and Ranging. *SAE International Journal of Connected and Automated Vehicles*. 4(12-04-04-0031): 413-422.

Jazwinski, A. H. (2007). *Stochastic processes and filtering theory*, Courier Corporation.

Julier, S. J. and J. K. Uhlmann (2004). Unscented Filtering and Nonlinear Estimation. *Proceedings of the IEEE*. 92(3): 401-422.

Jun, M. and A. J. Markel (2017). Infrastructure-based sensors augmenting efficient autonomous vehicle operations, National Renewable Energy Lab.(NREL), Golden, CO (United States).

Kaempchen, N., K. Weiss, M. Schaefer and K. C. Dietmayer (2004). IMM object tracking for high dynamic driving maneuvers. *IEEE Intelligent Vehicles Symposium, 2004*, IEEE.

Kim, S.-W., Z. J. Chong, B. Qin, X. Shen, Z. Cheng, W. Liu and M. H. Ang (2013). Cooperative perception for autonomous vehicle control on the road: Motivation and experimental results. *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE.

Liao, X., Z. Wang, X. Zhao, K. Han, P. Tiwari, M. J. Barth and G. Wu (2021). Cooperative ramp merging design and field implementation: A digital twin approach based on vehicle-to-cloud communication. *IEEE Transactions on Intelligent Transportation Systems*. 23(5): 4490-4500.

Luo, Y. (2019). Time constraints and fault tolerance in autonomous driving systems, Tech. rep, Tech. Rep.

Ni, Y., J. He, L. Cai and Y. Bo (2016). Delay analysis and message delivery strategy in hybrid V2I/V2V networks. *2016 IEEE Global Communications Conference (GLOBECOM)*, IEEE.

Orzechowski, P. F., A. Meyer and M. Lauer (2018). Tackling occlusions & limited sensor range with set-based safety verification. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, IEEE.

Petrillo, A., A. Salvi, S. Santini and A. S. Valente (2018). Adaptive multi-agents synchronization for collaborative driving of autonomous vehicles with multiple communication delays. *Transportation research part C: emerging technologies*. 86372-392.

Petrov, T., L. Sevcik, P. Pocta and M. Dado (2021). A performance benchmark for dedicated short-range communications and lte-based cellular-v2x in the context of vehicle-to-infrastructure communication and urban scenarios. *Sensors*. 21(15): 5095.

Polychronopoulos, A., M. Tsogas, A. J. Amditis and L. Andreone (2007). Sensor fusion for predicting vehicles' path for collision avoidance systems. *IEEE Transactions on Intelligent Transportation Systems*. 8(3): 549-562.

Schulz, D., W. Burgard, D. Fox and A. B. Cremers (2001). Tracking multiple moving targets with a mobile robot using particle filters and statistical data association. *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164)*, IEEE.

Thrun, S., W. Burgard and D. Fox (2006). Probabilistic robotics (intelligent robotics and autonomous agents series). *Intelligent robotics and autonomous agents, The MIT Press (August 2005)*.

Vu, T.-D. and O. Aycard (2009). Laser-based detection and tracking moving objects using data-driven markov chain monte carlo. *2009 IEEE international conference on robotics and automation*, IEEE.

Wan, E. A. and R. Van Der Merwe (2000). The unscented Kalman filter for nonlinear estimation. Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No. 00EX373), Ieee.

Wang, Z., K. Han and P. Han (2021). Motion estimation of connected and automated vehicles under communication delay and packet loss of V2X communications. *arXiv preprint arXiv:2101.07756*.

Wenzel, T. A., K. J. Burnham, M. V. Blundell and R. A. Williams (2006). Dual extended Kalman filter for vehicle state and parameter estimation. *Vehicle System Dynamics*. 44(2): 153-171.

Wu, J., H. Xu, Y. Zheng, Y. Zhang, B. Lv and Z. Tian (2019). Automatic vehicle classification using roadside LiDAR data. *Transportation Research Record*. 2673(6): 153-164.

Yin, F., D. Makris and S. A. Velastin (2007). Performance evaluation of object tracking algorithms. *IEEE International Workshop on Performance Evaluation of Tracking and Surveillance, Rio De Janeiro, Brazil*, Citeseer.

Zhang, L., J. Zheng, R. Sun and Y. Tao (2020). Gc-net: Gridding and clustering for traffic object detection with roadside lidar. *IEEE Intelligent Systems*. 36(4): 104-113.

Zhang, Z., J. Zheng, H. Xu and X. Wang (2019a). Vehicle detection and tracking in complex traffic circumstances with roadside LiDAR. *Transportation Research Record*. 2673(9): 62-71.

Zhang, Z., J. Zheng, H. Xu, X. Wang, X. Fan and R. Chen (2019b). Automatic background construction and object detection based on roadside LiDAR. *IEEE Transactions on Intelligent Transportation Systems*. 21(10): 4086-4097.

Zhao, J., H. Xu, H. Liu, J. Wu, Y. Zheng and D. Wu (2019). Detection and tracking of pedestrians and vehicles using roadside LiDAR sensors. *Transportation research part C: emerging technologies*. 10068-87.